SemGuard: Semantics-Decoupled Watermarking for Gaussian Splatting

Yingying Shi, Zhong Zhou, Bin Zhou

OVERVIEW

This supplementary material provides additional implementation details, training configurations, optimization settings, per-scene quantitative results, and qualitative analysis of SemGuard. The supplementary material is organized into four parts.

First, we describe the detailed implementation of semantic-robust decoder learning, including message representation, decoder architecture, and semantic interference curriculum.

Second, we present the Gaussian watermark embedding configurations for both 3DGS and 4DGS assets.

Third, we provide optimization details and training configurations to facilitate reproducibility.

Finally, we report additional quantitative and qualitative results, including per-scene evaluations and visual comparisons.

I. DETAILED DECODER LEARNING

A. Message Representation and Dataset Construction

During Stage-I decoder learning, we construct training samples consisting of a binary message, its CLIP-compatible textual representation, and an auxiliary semantic image. Given a binary message $m \in \{0,1\}^L$, we convert it into a token sequence $t_m = \mathrm{Tokenize}(m)$, which is encoded by the frozen CLIP text encoder:

$$F_T = E_T(t_m). \tag{1}$$

The corresponding target message m is used as the supervision signal for decoder training. To introduce semantic interference, we randomly sample an auxiliary image I_s and extract its CLIP visual feature:

$$F_I = E_V(I_s). \tag{2}$$

The auxiliary image is independent of the watermark message and only provides semantic variations during decoder learning. The semantic image pool contains rendered images from the static and dynamic Gaussian scene datasets described in the experimental setup.

Yingying Shi and Zhong Zhou are with the State Key Laboratory of Virtual Reality Technology and Systems, School of Computer Science and Engineering, Beihang University, Beijing, China, and also with Zhongguancun Laboratory, Beijing, China (e-mail: yyshi@buaa.edu.cn; zz@buaa.edu.cn).

Bin Zhou is with the State Key Laboratory of Virtual Reality Technology and Systems, School of Computer Science and Engineering, Beihang University, Beijing, China, and also with the Peace-Bay Academy of Virtual Reality, Shenyang, China (e-mail: zhoubin@buaa.edu.cn).

Corresponding author: Bin Zhou (e-mail: zhoubin@buaa.edu.cn).

B. Message Decoder Architecture

The message decoder is implemented as a lightweight multilayer perceptron to recover watermark bits from the fused CLIP feature. The decoder is optimized during Stage-I training, while the CLIP text encoder and image encoder remain frozen. After training, the decoder is shared across different Gaussian assets for watermark extraction.

C. Semantic Interference Weighting

The decoder input is constructed by combining watermark features and semantic features:

$$F_{mix} = (1-\rho)F_T + \rho F_I. \tag{3}$$

The interference weight ρ controls the contribution of semantic information. The continuous schedule adopted in SemGuard is:

$$\rho_e = \rho_{min} + (\rho_{max} - \rho_{min})\frac{e}{E}, \tag{4}$$

where e is the current epoch and E is the total number of training epochs. This curriculum gradually exposes the decoder to increasingly semantics-entangled representations.

II. GAUSSIAN WATERMARK EMBEDDING DETAILS

A. Optimized Gaussian Parameters

After Stage-I decoder learning, the semantic-robust decoder is frozen and used for Gaussian watermark embedding. Following the Gaussian watermarking pipeline, SemGuard embeds messages by optimizing spherical harmonic (SH) appearance coefficients. For each Gaussian primitive:

$$\hat{h}_i^k = h_i^k + \Delta h_i^k, \quad k \in \{dc, rest\}. \tag{5}$$

Only SH offsets are optimized, while Gaussian positions, scales, rotations, opacity, and deformation parameters (for 4DGS assets) remain unchanged. Therefore, the watermark is embedded into the Gaussian representation rather than a specific rendered image.

B. 3DGS Watermark Optimization

For static Gaussian assets, we randomly sample camera viewpoints and render watermarked views. The frozen CLIP image encoder extracts visual features:

$$F_V = E_V(I_w), \tag{6}$$

and the frozen semantic-robust decoder predicts watermark logits $o = D_\psi(F_V)$. The optimized SH offsets are updated according to the joint objective:

$$\mathcal{L} = \lambda_{msg}\mathcal{L}_{msg} + \lambda_{recon}\mathcal{L}_{recon} + \lambda_{off}\mathcal{L}_{off}. \tag{7}$$

TABLE I

PER-SCENE QUANTITATIVE RESULTS ON BLENDER AND LLFF 3DGS ASSETS. WE REPORT BIT ACC AND RENDERING FIDELITY METRICS (PSNR, SSIM, AND LPIPS) FOR DIFFERENT MESSAGE LENGTHS. THE SUPERSCRIPTS ⋆ AND † INDICATE THE SEMANTIC INTERFERENCE SOURCES USED DURING STAGE-I DECODER TRAINING: ⋆ USES STATIC SCENE IMAGES, WHILE † ADDITIONALLY INCORPORATES DYNAMIC SCENE IMAGES.

Methods	16 bits				32 bits				48 bits			
	Bit Acc	PSNR	SSIM	LPIPS	Bit Acc	PSNR	SSIM	LPIPS	Bit Acc	PSNR	SSIM	LPIPS
Blender [1]⋆												
Chair	100.00	47.87	0.9993	0.0004	99.77	44.66	0.9987	0.0007	99.14	41.08	0.9973	0.0014
Drums	100.00	44.15	0.9983	0.0008	97.34	39.88	0.9960	0.0018	95.72	39.17	0.9959	0.0020
Ficus	100.00	49.13	0.9995	0.0003	99.55	41.56	0.9979	0.0012	98.60	39.38	0.9969	0.0020
Hotdog	100.00	50.82	0.9993	0.0004	99.92	45.38	0.9981	0.0009	99.04	43.45	0.9972	0.0014
Lego	100.00	48.97	0.9992	0.0004	99.34	42.09	0.9969	0.0012	98.84	40.32	0.9955	0.0017
Materials	99.97	48.91	0.9994	0.0004	99.75	42.83	0.9977	0.0013	96.90	40.33	0.9960	0.0019
Mic	100.00	48.98	0.9995	0.0004	99.13	43.54	0.9985	0.0012	96.21	42.07	0.9979	0.0015
Ship	100.00	54.09	0.9997	0.0002	99.94	45.86	0.9982	0.0011	99.77	44.53	0.9978	0.0013
LLFF [2]⋆												
Fern	100.00	55.31	0.9996	0.0001	99.61	46.20	0.9981	0.0006	99.45	43.28	0.9971	0.0011
Flower	100.00	50.48	0.9991	0.0002	99.61	43.73	0.9964	0.0009	99.50	44.30	0.9965	0.0008
Fortress	99.88	50.43	0.9989	0.0004	99.70	44.56	0.9974	0.0009	99.58	43.03	0.9966	0.0011
Horns	100.00	51.60	0.9992	0.0003	99.50	44.00	0.9965	0.0015	99.36	43.74	0.9970	0.0013
Leaves	100.00	52.70	0.9996	0.0002	99.84	44.87	0.9979	0.0009	99.58	44.20	0.9978	0.0009
Orchids	99.81	44.82	0.9983	0.0005	99.02	39.11	0.9953	0.0013	99.06	39.43	0.9958	0.0013
Room	99.97	51.31	0.9992	0.0004	99.66	43.95	0.9969	0.0015	98.86	44.38	0.9968	0.0016
Trex	99.94	55.05	0.9998	0.0001	99.81	45.16	0.9983	0.0008	99.82	44.43	0.9981	0.0010
Blender [1]†												
Chair	99.88	45.20	0.9988	0.0007	99.25	41.52	0.9974	0.0014	98.59	41.33	0.9976	0.0013
Drums	99.88	42.23	0.9976	0.0013	98.73	40.03	0.9959	0.0019	91.77	39.77	0.9961	0.0019
Ficus	99.94	44.55	0.9988	0.0009	99.73	41.49	0.9978	0.0014	96.77	40.61	0.9975	0.0015
Hotdog	100.00	49.70	0.9991	0.0005	99.91	44.65	0.9977	0.0011	99.04	44.37	0.9975	0.0012
Lego	100.00	43.25	0.9976	0.0010	99.23	40.74	0.9957	0.0017	96.93	40.42	0.9953	0.0017
Materials	99.88	47.87	0.9991	0.0006	98.45	42.20	0.9971	0.0015	97.46	40.76	0.9964	0.0017
Mic	100.00	46.68	0.9993	0.0006	98.73	43.19	0.9984	0.0014	96.53	42.73	0.9982	0.0013
Ship	100.00	49.72	0.9993	0.0005	99.97	45.19	0.9979	0.0012	99.72	44.48	0.9976	0.0013
LLFF [2]†												
Fern	99.91	49.19	0.9990	0.0004	99.84	46.18	0.9980	0.0007	99.16	44.87	0.9976	0.0009
Flower	99.91	46.70	0.9978	0.0005	99.95	46.29	0.9978	0.0005	98.92	42.80	0.9956	0.0011
Fortress	99.97	49.26	0.9990	0.0004	99.73	45.12	0.9977	0.0008	98.69	44.03	0.9970	0.0011
Horns	99.94	46.66	0.9980	0.0008	99.69	46.30	0.9981	0.0010	98.89	43.09	0.9967	0.0016
Leaves	99.75	50.14	0.9992	0.0003	100.00	45.26	0.9984	0.0006	99.83	44.67	0.9977	0.0008
Orchids	99.81	42.65	0.9975	0.0007	99.16	40.46	0.9966	0.0010	98.04	39.18	0.9953	0.0014
Room	99.97	47.95	0.9984	0.0008	99.91	44.23	0.9972	0.0013	98.95	44.20	0.9965	0.0015
Trex	99.81	49.78	0.9993	0.0004	99.97	46.69	0.9986	0.0007	99.59	44.00	0.9979	0.0011

C. 4DGS Watermark Optimization

For dynamic Gaussian assets, we additionally sample temporal indices. Given a timestamp t and camera pose π, the rendered frame is $I_t = R(G, \pi, t)$. The same watermark message is enforced across sampled views and frames:

$$o_t = D_\psi(E_V(I_t)). \tag{8}$$

Therefore, the embedded watermark is encouraged to remain consistent under temporal deformation and appearance changes.

III. IMPLEMENTATION DETAILS

During Stage-I semantic-robust decoder learning, the message decoder is implemented as a lightweight MLP with the architecture $512 \to 256 \to 256 \to L$, where L denotes the message length. Each hidden layer uses GELU activation. The decoder is optimized during Stage-I semantic-robust decoder learning, while both CLIP encoders remain frozen. We evaluate message lengths of 16, 32, and 48 bits. The decoder is optimized using BCEWithLogits loss with a batch size of 64 and a learning rate of 5×10^{-3} using Adam optimizer. The number of sampled watermark messages is 2,048. For progressive semantic interference curriculum, the interference weight ρ is increased from $\rho_{min} = 0.0$ to $\rho_{max} = 0.9$.

For Stage-II Gaussian watermark embedding, the learned decoder and CLIP image encoder are frozen, and only spherical harmonic (SH) offsets are optimized. We use Adam optimizer with $\epsilon = 10^{-15}$. The DC SH coefficients are optimized with a learning rate of 5×10^{-3}, while the higher-order SH coefficients use a learning rate of $5 \times 10^{-3}/20$. The optimization objective consists of message recovery loss, reconstruction loss, and SH offset regularization, with $\lambda_{\text{recon}} = 1$ and $\lambda_{\text{off}} = 10$. For static 3DGS watermark embedding, we set $\lambda_{\text{msg}} = 0.03$ with a batch size of 16. For dynamic 4DGS watermark embedding, we set $\lambda_{\text{msg}} = 0.10$, sample 100 viewpoints and 8 temporal frames per viewpoint during each training iteration, and use a batch size of 8. All Gaussian geometry parameters remain fixed, and only SH offsets are

TABLE II
PER-SCENE QUANTITATIVE RESULTS ON D-NERF AND HYPERNERF 4DGS ASSETS. WE REPORT BIT ACC AND RENDERING FIDELITY METRICS (PSNR, SSIM, AND LPIPS) FOR DIFFERENT MESSAGE LENGTHS. THE SUPERSCRIPTS △ AND † INDICATE THE SEMANTIC INTERFERENCE SOURCES USED DURING STAGE-I DECODER TRAINING: △ USES DYNAMIC SCENE IMAGES, WHILE † USES BOTH STATIC AND DYNAMIC SCENE IMAGES.

Methods	16 bits				32 bits				48 bits			
	Bit Acc	PSNR	SSIM	LPIPS	Bit Acc	PSNR	SSIM	LPIPS	Bit Acc	PSNR	SSIM	LPIPS
D-NeRF [3]△												
Bouncingballs	100.00	44.09	0.9985	0.0009	96.94	40.29	0.9968	0.0015	86.85	39.04	0.9959	0.0018
Hellwarrior	99.88	45.75	0.9973	0.0013	94.66	44.46	0.9963	0.0017	89.54	43.26	0.9954	0.0016
Hook	99.94	45.01	0.9983	0.0010	95.91	41.68	0.9966	0.0015	98.04	41.89	0.9968	0.0013
Jumpingjacks	98.00	43.64	0.9976	0.0009	93.16	41.14	0.9956	0.0014	88.94	40.98	0.9962	0.0015
Lego	99.50	42.16	0.9966	0.0010	97.56	40.87	0.9955	0.0012	90.42	39.68	0.9944	0.0015
Mutant	99.88	44.77	0.9984	0.0008	98.91	41.35	0.9966	0.0011	99.08	43.36	0.9978	0.0008
Standup	99.19	40.29	0.9949	0.0016	97.66	40.83	0.9948	0.0015	91.40	41.41	0.9956	0.0012
Trex	99.81	46.49	0.9992	0.0007	98.94	45.37	0.9990	0.0009	90.73	43.52	0.9986	0.0013
HyperNeRF [4]△												
3Dprinter	78.81	48.95	0.9991	0.0007	78.69	45.52	0.9982	0.0013	78.60	44.49	0.9976	0.0016
Banana	93.50	50.99	0.9995	0.0005	92.38	45.38	0.9985	0.0014	92.02	44.19	0.9978	0.0016
Broom2	88.44	52.19	0.9997	0.0005	79.94	47.96	0.9991	0.0014	74.42	46.66	0.9986	0.0017
Chicken	83.19	49.57	0.9992	0.0005	87.47	45.50	0.9981	0.0014	90.54	44.91	0.9978	0.0015
D-NeRF [3]†												
Bouncingballs	100.00	43.66	0.9986	0.0007	99.78	40.77	0.9976	0.0013	92.50	39.77	0.9965	0.0014
Hellwarrior	99.69	46.93	0.9978	0.0013	98.41	45.75	0.9970	0.0013	82.71	44.28	0.9958	0.0016
Hook	100.00	45.95	0.9984	0.0009	99.78	45.73	0.9985	0.0009	95.38	41.79	0.9964	0.0016
Jumpingjacks	99.69	46.79	0.9989	0.0006	97.78	43.38	0.9975	0.0009	92.27	41.51	0.9963	0.0013
Lego	99.88	41.93	0.9966	0.0010	95.53	39.63	0.9943	0.0017	90.29	39.91	0.9943	0.0015
Mutant	99.88	44.85	0.9984	0.0009	99.59	45.60	0.9987	0.0006	95.75	41.27	0.9965	0.0013
Standup	99.88	44.19	0.9975	0.0011	99.25	42.52	0.9963	0.0010	89.90	40.08	0.9943	0.0016
Trex	99.56	48.87	0.9995	0.0005	98.69	46.50	0.9992	0.0007	95.00	44.89	0.9988	0.0009
HyperNeRF [4]†												
3DPrinter	86.94	49.14	0.9991	0.0006	84.44	46.13	0.9981	0.0011	71.67	44.88	0.9977	0.0015
Banana	95.50	51.00	0.9994	0.0005	97.09	47.04	0.9988	0.0011	92.42	43.75	0.9977	0.0018
Broom2	92.31	46.39	0.9987	0.0025	70.97	48.76	0.9992	0.0011	77.15	45.13	0.9984	0.0019
Chicken	88.75	50.99	0.9995	0.0003	88.38	46.42	0.9986	0.0009	86.69	45.94	0.9982	0.0011

updated. The SH offsets are optimized for 100 epochs, with checkpoint evaluation performed every 10 epochs. All experiments are conducted on NVIDIA A100 GPUs.

IV. MORE RESULTS OF ANALYSIS

A. Evaluation Protocol

For each Gaussian asset, watermark extraction is performed over multiple rendered viewpoints (and temporal frames for 4DGS). The reported scene-level metric is averaged over all rendered views. For 4DGS assets, temporal frames are additionally averaged within each sampled viewpoint. The final benchmark score reported in the main paper is computed by averaging scene-level metrics over the entire dataset.

B. Per-scene Quantitative Results

The results reported in the main paper are obtained by first averaging metrics over all rendered views of each scene and then averaging across all evaluated scenes. The following tables provide the corresponding per-scene measurements. The main paper reports the averaged performance over all evaluated scenes. To provide a more detailed analysis, we further report per-scene results for each individual 3DGS and 4DGS scene. These results demonstrate that SemGuard consistently achieves high watermark extraction accuracy across different scene categories, rather than relying on specific favorable scenes.

4DGS Results. Table II reports detailed results on D-NeRF and HyperNeRF scenes. Compared with static Gaussian assets, dynamic assets introduce additional challenges due to temporal deformation and appearance changes. For D-NeRF sequences, SemGuard maintains high Bit Accuracy across different motion patterns, demonstrating robustness against temporal variations. For HyperNeRF scenes, the performance varies across different sequences due to more complex appearance changes and deformation patterns. In addition, watermark embedding in dynamic Gaussian assets involves an inherent trade-off between extraction robustness and rendering fidelity. Increasing the message loss weight λ_{msg} can strengthen the embedded watermark signal and improve Bit Accuracy, but may introduce larger SH perturbations and reduce rendering quality. In this work, we adopt a unified $\lambda_{\mathrm{msg}} = 0.10$ for all 4DGS experiments to provide a consistent comparison protocol. Despite this constraint, SemGuard consistently outperforms GuardSplat on dynamic benchmarks, demonstrating the effectiveness of semantic-robust decoder learning. Comparing semantic interference strategies, SemGuard(△), trained only with dynamic semantic images, already provides strong robustness for dynamic assets. Adding static semantic sources in SemGuard(†) further improves performance for

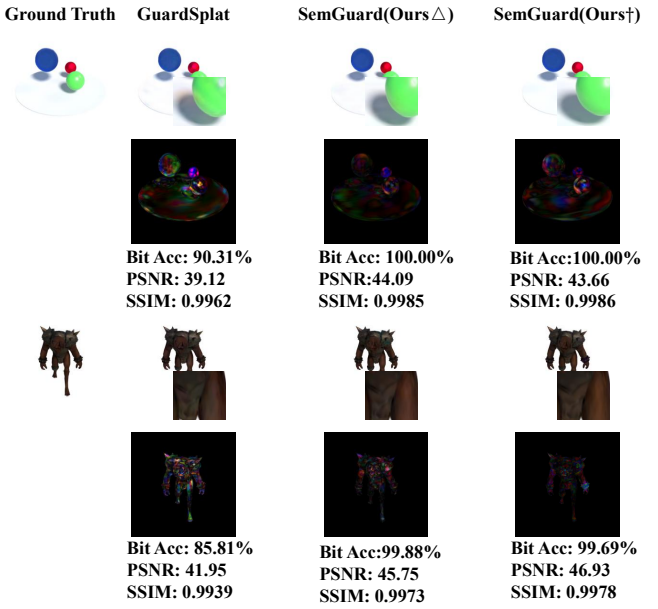


Fig. 1. Visual comparison on representative D-NeRF 4DGS scenes with 16-bit messages. We compare GuardSplat [5] with SemGuard variants trained with different semantic interference sources. The error maps visualize rendering differences, where darker regions indicate smaller perturbations.

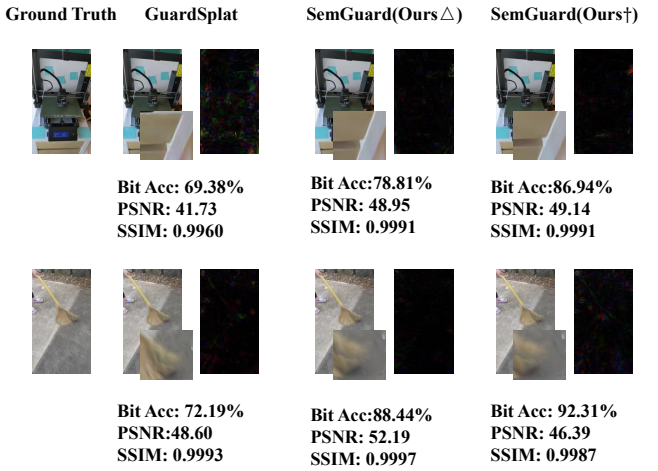


Fig. 2. Visual comparison on representative HyperNeRF 4DGS scenes with 16-bit messages. We compare GuardSplat [5] with SemGuard variants trained with different semantic interference sources. The error maps visualize rendering differences, where darker regions indicate smaller perturbations.

shorter messages, such as 16-bit messages, by introducing complementary semantic variations. These results demonstrate that diverse semantic interference distributions help learn a more transferable universal watermark decoder.

C. Additional Qualitative Results

To further demonstrate the visual fidelity and watermark extraction stability of SemGuard, we provide additional qualitative comparisons on static and dynamic Gaussian Splatting assets beyond the examples shown in the main paper. The rendered views and corresponding error maps are presented under different semantic interference sources. Figure **??** presents additional qualitative results on LLFF 3DGS scenes. Compared with GuardSplat [5], SemGuard variants preserve visual appearances closer to the original Gaussian assets while maintaining accurate watermark extraction. The smaller error regions indicate that semantic interference-aware decoder learning enables effective watermark embedding with fewer visual perturbations. Figures 1 and 2 further provide qualitative comparisons on dynamic Gaussian assets from D-NeRF and HyperNeRF. These results show that SemGuard maintains stable watermark extraction under temporal deformation and dynamic appearance variations. The consistent visual quality across different semantic interference sources demonstrates the generalization ability of the learned decoder beyond specific scene distributions.

ACKNOWLEDGMENTS

This work was supported in part by the National Natural Science Foundation of China (U25A20441), the National Science and Technology Major Project of China (2025ZD1700704), the National Key Research and Development Program of China (2024YFB3311703), and Zhongguancun Laboratory.

REFERENCES

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.

[2] B. Mildenhall, P. P. Srinivasan, R. Ortiz-Cayon, N. K. Kalantari, R. Ramamoorthi, R. Ng, and A. Kar, "Local light field fusion: Practical view synthesis with prescriptive sampling guidelines," *ACM Transactions on Graphics (ToG)*, vol. 38, no. 4, pp. 1–14, 2019.

[3] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer, "D-nerf: Neural radiance fields for dynamic scenes," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 10 318–10 327.

[4] K. Park, U. Sinha, P. Hedman, J. T. Barron, S. Bouaziz, D. B. Goldman, R. Martin-Brualla, and S. M. Seitz, "Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields," *arXiv preprint arXiv:2106.13228*, 2021.

[5] Z. Chen, G. Wang, J. Zhu, J. Lai, and X. Xie, "Guardsplat: efficient and robust watermarking for 3d gaussian splatting," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 16 325–16 335.